# 第10章 UNIX C 程序开发

詹文翰

## 内容简介


- 一、C 语言背景知识
- 二、GNU 工具链
- 三、编译过程
- 四、基于操作系统编程
- 五、调试
- 六、多文件编译




## 一、C 语言背景知识

- MULTICS 和 UNIX 的渊源
  - 1964年，在大型机时代，贝尔实验室、通用电气和麻省理工学院联合开发了 MULTICS 操作系统。 MULTICS 有一个宏大的目标，但实现过于复杂，并未达到设计目标。
  - 1969年，贝尔实验室退出该项目。K 神和 D 神吸取了 MULTICS 的经验和教训，开发出了 UNIX。可以认为，UNIX 是 MULTICS 的简化及升级。
  - UNIX 的第一个版本是完全用汇编语言编写的。



## 一、C 语言背景知识

- UNIX 和 C 语言的渊源
  - 当时，几乎所有操作系统都使用汇编语言进行开发。但 MULTICS 是一个例外，使用 PL/1 进行开发。其向世人展示了高级语言也可用于开发操作系统。K 神和 D 神也受其鼓舞，决定用高级语言重写 UNIX。
  - 最初他们选择 BCPL（Basic Combined Programming Language）。之后，又在 BCPL 的基础上创造了 B 语言。B 后来又被改进为 New B。最终，D 神又在 New B 的基础上创造了 C 语言。
  - 1973年，K 神和 D 神成功用 C 语言重写了 V3 UNIX。
  - 自此，UNIX 和 C 语言完美结合，成为世界的主导。现在，C 语言已经成为了操作系统开发最常用的编程语言。



## 一、C 语言背景知识

- C 语言的特点
  - 结构化程序设计语言
  - 简洁紧凑，方便灵活
  - 功能强大，具有丰富的数据类型及运算符
  - 语法限制不太严格，程序设计自由度大
  - 允许直接访问内存地址，能进行位操作，具有汇编语言的功能
  - 编译型语言（而非解释型），生成目标代码质量高，程序执行效率高
  - 可移植性好（相对汇编语言），源代码级的可移植性



## 一、C 语言背景知识

- C 语言标准
  - 自 1972 年诞生以来，C 语言的流行度越来越高。众多公司基于其对 C 语言的理解自行开发编译器，却没有统一标准，导致 C 语言方言越来越多。
  - 1983年，ANSI 为 C 语言制定了统一标准，称为 ANSI C，并于 1989 年最终完成（iso 9899:1990）。
    - ANSI C 于 1989 年通过审批，1990 年发布。因此，该标准的以下称呼等价：
    - **ANSI C == C89 == C90**
  - 1995 年，针对 90 年发布的 ANSI C 标准，又发布了一个修订版（iso 9899:199409）。
    - 该修订版可被称作：**AMD1 == C94 == C95**
    - 轻微修订：增加了宏 __STDC_VERSION__，进行国际化
    - **该版本为我国现行的 C 语言标准**



## 一、C 语言背景知识

- C 语言标准
  - 1999 年，新的 C 语言标准（ISO/IEC 9899:1999）发布：
    - 该标准常被叫做：**C99**
    - C99 标准未完成之前的草案，被叫做：C9X
    - C99 后来在 2001 年，2004 年和 2007 年，先后发布过对应的勘误表
    - 增加了部分库函数，以及对复数的支持
  - 2011 年，C 语言又发布了新标准：ISO/IEC 9899:2011
    - 该标准常被叫做：**C11**
    - C11 标准未完成之前的草案，被叫做：C1X
  - 开源组织 GNU 也单独制定了 C 语言的标准： GNU C
    - 对上述各版 C 语言标准，GNU C 都进行了一定的扩展。



## 一、C 语言背景知识

- C 语言标准总结

| 标准名 | C89 | AMD1 | C99 | C11 |
|---|---|---|---|---|
| 别名 | C90<br>ANSI C<br>X3.159-1989<br>ISO/IEC 9899:1990 | C94<br>C95<br>ISO/IEC 9899:199409 | ISO/IEC 9899:1999 | ISO/IEC 9899:2011 |
| 发布时间 | 1990年 | 1995年 | 1999年 | 2011年 |
| GCC 使用此标准所需参数 | -ansi<br>-std=c90<br>-std=iso9899:1990 | -std=iso9899:199409 | -std=c99<br>-std=iso9899:1999 | -std=c11<br>-std=iso9899:2011 |
| GCC 使用此标准且带 GNU 扩展时所需参数 | **-std=gnu90（默认）** | | -std=gnu99 | -std=gnu11 |
| 标准实现时所增加的函数库 | <float.h><br><limits.h><br><stdarg.h><br><stddef.h> | <iso646.h> | <stdbool.h><br><stdint.h> | <stdalign.h><br><stdnoreturn.h> |



## 一、C 语言背景知识

- C 语言开发环境
  - C 语言开发环境众多，包括纯命令行环境和集成开发环境（IDE）。
  - 曾经和当前较为流行的开发环境包括：
    - ANSI C
    - Borland C
    - Turbo C
    - Dev C++
    - Microsoft Visual C++
    - Microsoft Visual Studio
    - Clang
    - GNU GCC
    - CLion
  - 理清以下几个环境：
    - 编辑环境、编译环境、集成开发环境、运行环境

# 二、GNU 工具链

- GNU toolchain 是开发操作系统和应用程序的一套完整的工具集，类似于工业制造中的母机。
- GNU toolchain 中，和 C/C++ 语言开发相关的工具包括：
  - cpp
  - gcc/g++
  - glibc/libc++
  - gdb
  - binutils：as、ld、ar、objdump、nm、c++filt、gprof、readelf...
  - make
  - autotools：autoconf、automake、libtool...



# 二、GNU 工具链

- GCC 有两层含义：
  - GNU Compiler Collection：是一族编译器，包括 C、C++、Objective-C、Java等。
  - GNU C Compiler：专指 GNU 的 C 语言编译器。
- GCC 支持的文件类型：
  - .c　C 源代码文件；
  - .h　C 头文件；
  - .i　经预处理后的 C 源代码文件；
  - .s　汇编语言源代码文件；
  - .o　编译后的目标文件；
  - .a　静态链接库文件；
  - .so　动态链接库文件；



# 三、编译过程

- C 语言编译过程：预处理 -> 编译 -> 汇编 -> 链接 -> 加载运行
- gcc 编译过程拆解：
  - gcc -o hello.i -E hello.c
  - gcc -o hello.s -S hello.i
  - gcc -o hello.o -c hello.s
  - gcc -o hello hello.o

demo

# 三、编译过程

- gcc 基本用法：
  - gcc [选项] 文件名列表
  - 文件名列表 指的是需要预处理、编译、汇编和链接的文件。
  - gcc 支持的 [选项] 很多，参见 man，这里列出常用选项。
  - gcc 举例：
    - 多文件编译
    - 链接库

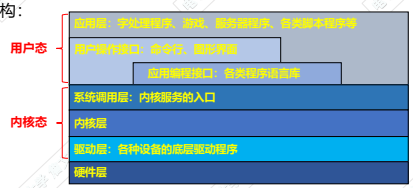
demo

| 选项 | 说明 |
|---|---|
| -E | 在预处理器执行完成之后停止，不编译 |
| -S | 在编译器执行完成之后停止，不汇编 |
| -c | 在汇编器执行完成之后停止，不链接 |
| -o outfile | 将输出结果写入 outfile 文件中 |
| -v | 显示详细的编译过程 |
| -g | 生成调试信息，供调试器（gdb）使用 |
| -DMACRO[=DEFINE] | 定义宏，默认将宏定义为 "1" |
| -IDIRECTORY | 指定额外的头文件搜索路径 |
| -LDIRECTORY | 指定额外的函数库搜索路径 |
| -library | 链接时包含指定的函数库 |
| -fpic 或 -fPIC | 生成位置无关代码 |
| -shared | 生成共享目标文件，通常用于创建共享库 |
| -static | 链接时禁止使用共享库 |
| -Wall | 显示在编译过程中的所有警告信息 |
| -w | 不显示任何警告 |
| -Olevel | 按照优化等级 level 进行编译优化 |
| -pg | 为性能分析工具 gprof 生成额外代码 |
| -std=standard | 指定语言标准，参考 "C 语言标准总结" |
| -ansi | 等价于 -std=c90 或 -std=c++98 |



# 四、基于操作系统编程

- 操作系统：是资源的管理和保护者，并对上层用户提供友好的编程环境（连续、隔离、独占的内存空间）和统一的资源访问接口（系统调用）。
- 操作系统的层次结构：

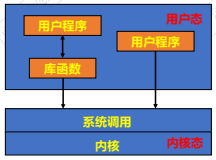





# 四、基于操作系统编程

- 用户程序通过系统调用来实现对内核功能的使用和对硬件资源的访问
- 系统调用按功能分类：
  - 输入、输出控制类
  - 进程、线程管理类
  - 内存管理类
  - 设备控制类
  - 进程间通信类（本地/网络）

demo




- 系统调用与库函数的关系
  - 系统调用是操作系统层面提供的服务，库函数是高级语言提供的服务



# 五、调试

- GDB（GNU Debugger）是 GNU 工具链提供的一个功能强大的调试器，用于调试和分析源代码级别的错误。其主要功能和特点包括：
  - 断点调试：允许在程序执行过程中设置断点，包括行级断点、函数级断点和条件断点。
  - 单步执行：允许逐行或逐指令执行程序，可以单步跟踪程序的执行流程，观察变量的值和状态的变化，以便找出问题所在。
  - 变量和内存查看：允许对变量、内存地址和寄存器中的数据进行查看和监视。
  - 回溯和堆栈跟踪：当程序发生崩溃或异常时，GDB 可提供堆栈跟踪信息，显示函数调用堆栈，帮助定位错误和异常。
  - 远程调试：GDB 支持连接到运行在不同机器或嵌入式设备上的程序进行远程调试。



# 五、调试

- gdb 基本用法：
  - 格式：gdb [ 可执行文件 | 可执行文件 PID | 可执行文件 core ]
  - 注意：
    - gdb 程序调试的对象是可执行文件，而不是程序的源代码文件。
    - 为方便 gdb 对可执行文件进行调试，在进行编译时需为 gcc/g++ 加上 -g 参数，用于在二进制目标文件中包含调试信息。gdb 利用这些调试信息进行源代码和机器码之间的关联。
    - 启动 gdb 时可以暂不指定可执行文件，而是在 gdb 中通过 file 命令指定。



# 五、调试

- gdb 常用命令——基本：
  - 设置可执行文件：
    - file 可执行文件
  - 界面布局：
    - layout split | src | asm | regs
  - 执行 shell 命令：
    - shell 命令
  - 帮助：
    - help
  - 退出 gdb（quit）：
    - q

## 五、调试

- gdb 常用命令——断点：
  - 打断点（break/watch）：
    - b [[文件:]行号] [if 条件]
    - watch 表达式
  - 查看断点（info）：
    - i b [断点编号]
  - 删除断点（delete）：
    - delete [断点编号 | 断点范围]
  - 使能断点（enable/disable）：
    - enable [断点编号 | 断点范围]
    - disable [断点编号 | 断点范围]



## 五、调试

- gdb 常用命令——调试：
  - 启动（run）：
    - r
  - 暂停（<Ctrl-c>）：
    - <Ctrl-c>
  - 停止（kill）：
    - kill
  - 显示当前执行位置（where）：
    - where



## 五、调试

- gdb 常用命令——调试：
  - 单步进入（step/stepi）：
    - s
    - si
  - 单步（next/nexti）：
    - n
    - ni
  - 跳出当前函数（finish）：
    - finish
  - 继续执行（continue）：
    - c



## 五、调试

- gdb 常用命令——查看：
  - 查看源码（list）：
    - l [[文件名:]行号]
      - 设置/显示查看的行数：set/show listsize 行数
  - 查看变量（print/display）：
    - p [文件名:]变量名
      - 设置友好显示：set print pretty
    - display 表达式
      - 每次停止运行时均显示 表达式 的值
  - 查看变量类型（ptype）：
    - ptype 变量名



## 五、调试

- gdb 常用命令——查看：
  - 查看函数堆栈（backtrace）：
    - bt [[-]栈帧数量]
  - 查看栈帧（frame）：
    - f [帧编号]
  - 在栈帧间移动（up/down）：
    - up [栈帧数量]
    - down [栈帧数量]
  - 查看当前栈帧局部变量（info）：
    - i locals



## 五、调试

- gdb 常用命令——修改：
  - 修改变量（set）：
    - set variable 变量名 = 表达式
    - 例如：
      - set variable i = 10
  - 修改内存内容（frame）：
    - set (强制类型转换)地址 = 表达式
    - 例如：
      - set (int)0x83040 = 4

demo



## 六、多文件编译

- 实际的 C/C++ 工程项目均由大量源文件构成。对此类项目的构建需要对项目中各个源文件进行分别编译，之后再进行链接。手工完成该过程会对用户带来极大的不便。
- gcc 支持一次性编译多个源文件并完成链接工作，但该方式对大型程序的开发和构建并不友好：
  - 仅能全量构建，不支持增量构建
  - 缺乏依赖管理，对开发者不友好
  - 仅能串行构建，构建时间过长



## 六、多文件编译

- GNU make
  - make 是 GNU 工具链提供的一款广受欢迎的程序构建工具，提供了丰富的功能来管理和组织源代码的编译、链接和其它构建任务。其主要的特点和功能包括：
    - makefile：make 使用 makefile 来描述项目的构建规则和依赖关系，用以进行自动化构建。
    - 自动化构建：make 可以基于 makefile 自动检测源文件和依赖项是否发生更新，并根据依赖关系自动决定需要重新编译的文件，以避免不必要的重新编译，提高构建效率。
    - 并行编译：make 支持并行编译（-j 参数），充分利用多核性能并行编译多个源文件，加快构建速度。
    - 丰富的变量和函数：make 提供了丰富的内置变量和函数，用于方便地操作和管理构建过程。这些变量和函数可以用于指定编译器、编译选项、目标文件名等，以及进行文件操作和字符串处理等常用任务。
    - 多语言支持：make 工具是语言无关的，可用于构建任何语言编写的软件（只要提供命令行编译器）。



## 六、多文件编译

- make 使用方法：
  - 使用 make 构建程序时必须为其提供 makefile 文件。默认情况下，make 会在当前工作目录中按如下文件名顺序搜索 makefile：
    - GNUmakefile -> makefile -> **Makefile**
  - 只要 makefile 文件存在，则可以直接运行 make 开始程序构建：

```
make
```

  - 也可以使用 -f 参数来指定其它文件作为 makefile，如：

```
make -f Makefile.debug
```

# 六、多文件编译

- makefile
  - makefile 是由一系列规则（rules）所构成的编译说明文件。
  - 与常见的命令式语言不同，它是一种依赖推导语言：
    - shell：变量定义+命令执行
    - Makefile：变量定义+依赖描述
  - 基本格式：

```
target : prerequisties
<Tab>commands
```

  - target：目标，可以是对象文件、可执行文件、标签（也被称作“伪目标”）
  - prerequisite：实现该 target 所依赖的文件或其它 target，以空格分割
  - commands：实现该 target 所需执行的（shell）命令，可以有多行

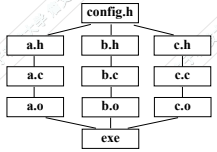


# 六、多文件编译

- makefile 示例

```
exe: a.o b.o c.o
        gcc -o exe a.o b.o c.o
a.o: a.c a.h config.h
        gcc -c a.c
b.o: b.c b.h config.h
        gcc -c b.c
c.o: c.c c.h config.h
        gcc -c c.c
clean:
        rm -f *.o
```

伪目标




- 构建程序时，执行： make [target]
  - 若没有指定 [target]，make 会将 makefile 中第一个 target 作为构建目标。
  - make 会根据 makefile 中指定的依赖关系，从 target 出发，基于动态规划递归地向下寻找，直到枚举出构建 target 完整的依赖关系。之后，再根据该依赖关系逐条执行规则中的对应 commands。
  - 如依赖关系（图）中有任何文件发生修改，则再次执行 make 时并不会重新构建与该修改无关的项目。



# 六、多文件编译

demo

- makefile 变量和函数
  - makefile 支持变量和函数，以方便在构造规则时进行复杂**字符串处理**。
  - 常用变量：
    - $@ 目标的名称
    - $^ 所有的依赖名称，以空格分开
    - $< 第一个依赖的名称

| 选项 | 说明 |
|---|---|
| -f otherfile | 使用 otherfile 作为 makefile 文件 |
| -j NUM | 并行构建，最多 NUM 路 |
| -p | 显示默认的 makefile 变量和隐式规则 |
| -s | 构建过程中不显示 commands |

  - 常用函数：
    - $(shell <command>)
    - $(subst <old>,<new>,<text>) 和 $(patsubst <pattern>,<new_pattern>,<text>)
    - $(dir <names...>) 和 $(notdir <names...>)



# 第十章小结

- C 语言背景知识
- GNU 工具链
- 编译过程
- 基于操作系统编程
- 调试
- 多文件编译